

ProbaLean: A Probabilistic DSL in Lean

Jamie Fulford*
jamiehf2@illinois.edu
University of Illinois
Urbana-Champaign
Urbana, IL, USA

Trey Plante*
eplante3@illinois.edu
University of Illinois
Urbana-Champaign
Urbana, IL, USA

Zirui Zhou*
zirui8@illinois.edu
University of Illinois
Urbana-Champaign
Urbana, IL, USA

1 Introduction

Probabilistic programming languages (PPLs) allow statistical models to be expressed as ordinary programs augmented with primitives for sampling from distributions and conditioning on observations. The appeal of this approach is that inference algorithms can be written once, generically, and applied to any model expressed in the language. Despite this convenience, establishing the correctness of a probabilistic program—that its operational behavior matches the intended probability distribution—remains difficult to achieve informally.

A principled solution requires a **denotational semantics**: a compositional mapping from programs to mathematical objects (typically measures or kernels for probabilistic programs) that can be reasoned about independently of any execution strategy. The Giry monad [3], which equips the category of measurable spaces with a probability-measure functor, provides the canonical semantic domain. More recent work has scaled this foundation to richer language features: Staton [4] identifies the commutativity of independent sampling—the fact that drawing from two independent distributions in either order induces the same joint measure—as a key algebraic property of probabilistic programs, and proves it for a language with soft conditioning via a score primitive. Affeldt et al. [2] carry out a related formalization in Rocq using s-finite kernels, demonstrating that the approach is amenable to machine-checked proof.

In our project, we present **ProbaLean**¹, an embedding of a discrete PPL in Lean 4 with fully machine-checked denotational semantics built on Mathlib [1]. ProbaLean is designed around two semantic layers. The *operational* layer interprets programs as weighted lists of outcomes, enabling direct execution and interactive experimentation. The *denotational* layer interprets programs as s-finite measures over their return types, grounding the language in standard measure theory. A soundness theorem formally connects the two: the measure induced denotationally agrees with the measure induced by the operational enumeration, for any well-behaved conversion between the two weight representations. A key design choice is to parameterize programs by their weight type, so that the same syntactic program can

be evaluated concretely or interpreted formally without duplication.

2 Motivating Example

This section introduces the central running example—Bayesian inference for a medical test—to give a concrete sense of what ProbaLean provides: a surface language for expressing probabilistic models, an executable operational semantics, and a machine-checked denotational semantics that proves the program’s output correct by Bayes’ theorem.

2.1 The Medical Test Problem

Consider a rare disease with prevalence 1% in the general population. A diagnostic test for this disease has sensitivity 95%—it returns a positive result on 95% of true cases—and a false-positive rate of 10%—it returns a positive result on 10% of healthy individuals. Given that a patient tests positive, what is the probability that the patient actually has the disease?

By applying Bayes’ theorem directly, we get:

$$\begin{aligned} P(\text{disease} \mid \text{test}^+) &= \frac{P(\text{test}^+ \mid \text{disease}) P(\text{disease})}{P(\text{test}^+)} \\ &= \frac{0.95 \times 0.01}{0.95 \times 0.01 + 0.10 \times 0.99} = \frac{0.0095}{0.1085} \approx 8.8\%. \end{aligned}$$

The result is strikingly low: even after a positive test, the probability of disease is only about 1 in 11. The reason is that false positives—affecting 10% of the healthy 99%—vastly outnumber true positives from the rare 1%. This base-rate effect is a canonical illustration of why probabilistic reasoning is difficult to do correctly without formal tools.

2.2 The ProbaLean Program

ProbaLean provides a `prob { ... }` surface notation in which probabilistic models are written as sequential programs. The medical test is expressed as:

```
def medicalTest : Prog Float Bool := prob {  
  disease ~ bernoulli 0.01;  
  testPos ~ bernoulli (if disease then 0.95  
    else 0.10);  
  observe testPos;  
  return disease  
}
```

Each statement in the block corresponds to one step of the model. The first line samples a Boolean disease from a Bernoulli distribution with parameter 0.01. The second line

*All authors contributed equally to this project.

¹<https://github.com/trey3p/probalean>

samples `testPos` from a Bernoulli distribution whose parameter depends on disease, encoding the sensitivity and false-positive rate. The observe statement hard-conditions on the test being positive: any execution branch in which `testPos` is false is discarded. Finally, the block returns `disease`, so the program’s output is the posterior distribution over the disease status given a positive test result.

Evaluating the program with the built-in operational semantics yields the posterior directly:

```
#eval medicalTest.run
-- [(true, 0.087557), (false, 0.912442)]
```

The weight at `true` is approximately 0.0876, confirming the analytical calculation. Beyond producing the correct numerical answer, ProbaLean allows the computation to be proved correct. The central correctness theorem establishes that this measure obtained from the ProbaLean program is equal to the joint distribution prescribed by Bayes’ rule.

```
theorem medicalTestE_posterior_true :
  medicalTestE.evidence * medicalTestE.normalize
  ({true} : Set Bool) = (95 / 10000 : ENNReal)
```

3 Mathematical Preliminaries

This section reviews the measure-theoretic foundations on which ProbaLean’s denotational semantics rests.

A σ -algebra on a set Ω is a collection $\Sigma \subseteq \mathcal{P}(\Omega)$ that contains Ω , is closed under complementation, and is closed under countable union. The pair (Ω, Σ) is called a *measurable space*, and elements of Σ are *measurable sets*. A function $f : (\Omega_1, \Sigma_1) \rightarrow (\Omega_2, \Sigma_2)$ is *measurable* if for any measurable set $B \in \Sigma_2$, $f^{-1}(B)$ is also measurable, which is $f^{-1}(B) \in \Sigma_1$.

A *measure* on a measurable space (Ω, Σ) is a function $\mu : \Sigma \rightarrow [0, \infty]$ that satisfies $\mu(\emptyset) = 0$ and for any countable collection of pairwise-disjoint sets $\{A_i\}$, we have $\mu(\cup_i A_i) = \sum_i \mu(A_i)$. Two measures play a special role. The *zero measure* assigns mass 0 to every set; it arises as the denotation of a conditioning step on a false observation. The *Dirac measure* δ_p at a point a assigns mass 1 to every set containing p and mass 0 to all others; it serves as the denotation of a pure return value. Together, these form the unit and the “atom” of the measure monad.

A measure μ is *finite* if $\mu(\Omega) < \infty$, and *σ -finite* if Ω can be covered by countably many sets of finite measure. A measure is *s-finite* if it can be written as a countable sum of finite measures [2]. Every σ -finite measure is s-finite, making s-finiteness a broad and technically convenient generalization. Unlike σ -finite measures, s-finite measures are closed under arbitrary countable sums, which simplifies compositional reasoning.

A *kernel* from a measurable space (Γ, Σ_Γ) to (α, Σ_α) is a function $\kappa : \Gamma \rightarrow \text{Measure}(\alpha)$ such that for every measurable set $S \subseteq \alpha$, the function $\gamma \mapsto \kappa(\gamma)(S)$ is measurable. Kernels generalize conditional distributions: they describe how the measure over outcomes depends on the current state.

We can integrate a kernel over a measure by *measure bind*. Given a measure μ on β and a kernel $\kappa : \beta \rightarrow \text{Measure}(\alpha)$, the measure bind $\mu \gg \kappa$ is defined by

$$(\mu \gg \kappa)(S) = \int_{\beta} \kappa(b)(S) d\mu(b).$$

Intuitively, $\mu \gg \kappa$ models two-stage random sampling: a value b is first drawn according to μ , and then a second value is drawn from the measure $\kappa(b)$, which may depend arbitrarily on b . The result is a single measure on α that marginalizes over the intermediate value b .

The construction $\alpha \mapsto \text{Measure}(\alpha)$, equipped with the unit $\eta_\alpha(a) = \delta_a$ and the bind operation mentioned above, forms a monad in the category of measurable spaces—the *Giry monad* [3]. The unit η embeds pure values into the probabilistic setting: δ_a is the deterministic distribution that returns a with certainty. The bind $\mu \gg \kappa$ then captures the sequencing of a probabilistic computation μ followed by a continuation κ that may itself be probabilistic. Together, they provide the two building blocks of any compositional probabilistic language: deterministic return and probabilistic sequencing.

The *Kleisli category* of the Giry monad has measurable spaces as objects and kernels $\kappa : \alpha \rightarrow \text{Measure}(\beta)$ as morphisms. Sequential composition of two Kleisli morphisms $\kappa : \alpha \rightarrow \text{Measure}(\beta)$ and $\lambda : \beta \rightarrow \text{Measure}(\gamma)$ is given by $(\kappa ; \lambda)(a) = \kappa(a) \gg \lambda$, which reduces to measure bind applied pointwise. Every probabilistic program can be interpreted as a morphism in this category, and the monad laws precisely ensure that this interpretation is compositional.

4 Mathlib Preliminaries

The weight type for the denotational semantics is `ENNReal`, Mathlib’s type for the extended non-negative reals $[0, \infty]$. `ENNReal` supports the full arithmetic needed for measure theory without the partiality or overflow issues that arise with `Float` or $\mathbb{R}$. The top element $\top = \infty$ is essential for representing the unnormalized weights produced by the score primitive, which may be infinite. Subtraction on `ENNReal` is saturating ($a - b = 0$ when $b \geq a$).

The typeclass `MeasurableSpace` α equips a type α with a σ -algebra, making it a formal measurable space. The companion typeclass `MeasurableSingletonClass` α asserts that every singleton $\{a\}$ is a measurable set; this holds for any type with the discrete σ -algebra.

Mathlib represents measures as the type `MeasureTheory.Measure` α . A term $\mu : \text{Measure } \alpha$ is a function from measurable sets to `ENNReal` satisfying the standard measure axioms. The type is a complete lattice under pointwise ordering, and supports scalar multiplication, addition, and countable sums. `MeasureTheory.dirac a : Measure` α is the Dirac measure at point a , assigning mass 1 to any measurable set containing a and 0 otherwise.

A kernel κ from Γ to α is a measurable family of measures: for each $\gamma : \Gamma$, a measure $\kappa \gamma : \text{Measure } \alpha$, such that $\gamma \mapsto \kappa \gamma S$ is measurable for every measurable set S . Mathlib’s type `ProbabilityTheory.Kernel` $\Gamma \alpha$ packages this data together with the measurability proof. We mention two specific kernels here: (1) `Kernel.const` $\Gamma \mu$ is the constant kernel that ignores its argument and always returns the fixed measure μ . (2) `Kernel.ofFunOfCountable` f constructs a kernel from a function $f : \beta \rightarrow \text{Measure } \alpha$ when β is countable. This is useful when we do sampling on a discrete space.

`Measure.bind` $\mu \kappa : \text{Measure } \alpha$ is Mathlib’s realization of the measure-theoretic bind from Section 3. Given a measure $\mu : \text{Measure } \beta$ and a kernel $\kappa : \text{Kernel } \beta \alpha$, it computes $(\mu \gg \kappa)(S) = \int_{\beta} \kappa(b)(S) d\mu(b)$. The key computational lemma used in ProbaLean’s proofs is `Measure.comp_eq_sum_of_countable`, which states that when β is countable, $\mu \gg \kappa = \sum_{b:\beta} \kappa(b)$.

5 Syntax of Probabilistic Programming Language

5.1 Core Language

This section presents the abstract syntax and type system of the core ProbaLean language.

5.1.1 Types. ProbaLean distinguishes two classes of types. *Weight types* P carry probability annotations and determine the numeric representation used for evaluation or proof. *Value types* α are the ordinary Lean types that programs return.

$$P ::= \text{Float} \mid \mathbb{Q}_{\geq 0} \mid \text{ENNReal}$$

$$\alpha, \beta ::= \text{Bool} \mid \text{Fin } n \mid \text{Unit} \mid \alpha \times \beta \mid \dots$$

The weight type is a parameter of the language, not a fixed choice: the same program term can be instantiated at any P . In practice, `Float` is used for concrete execution, `$\mathbb{Q}_{\geq 0}$` for exact computable arithmetic, and `ENNReal` for denotational proof.

The syntactic category $\text{Dist } P \alpha$ names the primitive distributions. Each constructor is indexed by its return type α .

$$d ::= \begin{array}{ll} \text{bernoulli } p & (p : P) \\ \mid \text{uniform_fin } n & (n : \mathbb{N}) \\ \mid \text{categorical } f & (f : \text{Fin } n \rightarrow P) \end{array}$$

The `bernoulli` p constructs a Bernoulli distribution with a random variable that takes value `true` with probability p and `false` with probability $1 - p$. The `uniform_fin` n constructs a discrete uniform distribution in which a random variable has n outcome values each having probability $1/n$. The `categorical` f generalizes `uniform_fin` n by assigning the outcome k with weight $f(k)$.

5.1.2 Programs. The syntactic category $\text{Prog } P \alpha$ denotes programs that, when run, produce a value of type α weighted by elements of P . The grammar has four constructors; sequencing (‘bind’) is a derived operation defined by structural recursion and is not a constructor.

$$e ::= \begin{array}{ll} \text{ret } v & (v : \alpha) \\ \mid \text{sample } d (\lambda x. e) & (d : \text{Dist } P \beta, x : \beta) \\ \mid \text{observe } b e & (b : \text{Bool}) \\ \mid \text{score } r e & (r : P) \end{array}$$

The derived sequencing form is:

$$e \gg= (\lambda x. e') \quad (\text{bind, by recursion on } e)$$

The typing rules for programs are:

$$\begin{array}{c} \frac{\Gamma \vdash v : \alpha}{\Gamma \vdash \text{ret } v : \text{Prog } P \alpha} \quad (\text{T-Ret}) \\ \frac{\Gamma \vdash d : \text{Dist } P \beta \quad \Gamma, x : \beta \vdash e : \text{Prog } P \alpha}{\Gamma \vdash \text{sample } d (\lambda x. e) : \text{Prog } P \alpha} \quad (\text{T-Sample}) \\ \frac{\Gamma \vdash b : \text{Bool} \quad \Gamma \vdash e : \text{Prog } P \alpha}{\Gamma \vdash \text{observe } b e : \text{Prog } P \alpha} \quad (\text{T-Obs}) \\ \frac{\Gamma \vdash r : P \quad \Gamma \vdash e : \text{Prog } P \alpha}{\Gamma \vdash \text{score } r e : \text{Prog } P \alpha} \quad (\text{T-Score}) \\ \frac{\Gamma \vdash e : \text{Prog } P \alpha \quad \Gamma, x : \alpha \vdash e' : \text{Prog } P \beta}{\Gamma \vdash e \gg= (\lambda x. e') : \text{Prog } P \beta} \quad (\text{T-Bind}) \end{array}$$

Variable binding in `sample` is represented using Higher-Order Abstract Syntax (HOAS): the continuation is an ordinary Lean function of type $\beta \rightarrow \text{Prog } P \alpha$. Variable scoping is handled entirely by Lean’s own function type, avoiding the need for de Bruijn indices or explicit substitution. As a consequence, `bind` is not a constructor of `Prog` but a defined function by structural recursion:

$$\begin{array}{ll} (\text{ret } v) \gg= k & = k v \\ (\text{sample } d c) \gg= k & = \text{sample } d (\lambda x. c x \gg= k) \\ (\text{observe } b e) \gg= k & = \text{observe } b (e \gg= k) \\ (\text{score } r e) \gg= k & = \text{score } r (e \gg= k) \end{array}$$

Together with `ret` as the unit, this definition makes `Prog` P a monad in Lean’s `Monad` typeclass, which in turn enables standard `do`-notation.

Another notable thing here is the *Weight-Type Coercion*. A function $f : P \rightarrow Q$ lifts to a homomorphic map on programs:

$$\frac{\Gamma \vdash f : P \rightarrow Q \quad \Gamma \vdash e : \text{Prog } P \alpha}{\Gamma \vdash \text{coerce } f e : \text{Prog } Q \alpha} \quad (\text{T-Coerce})$$

Coercion acts homomorphically: it replaces every weight annotation in e by its image under f without altering the continuation structure or control flow. It is the mechanism by which a program written for one weight type (e.g., `Float`) is transported to another (e.g., `ENNReal`) for formal reasoning.

5.2 Surface Syntax

ProbaLean provides two complementary surface interfaces, both implemented as Lean macros that desugar to the core language.

5.2.1 Monadic Do-Notation. Because $\text{Prog } P$ is a Monad , Lean’s standard do-notation is available directly. With several helper defined combinators, programs written in this style look like ordinary imperative code:

```
def biasedHeads : Prog Float Bool := do
  let x ← sample (bernoulli 0.7)
  condition x
  return x
```

5.2.2 Prob-Block Notation. For users who prefer a notation closer to mathematical model specification, ProbaLean provides a dedicated ‘prob ...’ block syntax. Its grammar is:

```
stmt ::= x ~ d   (sample from distribution d)
      | let x ← e (bind from program e)
      | let x := v (pure let-binding)
      | observe b (hard conditioning)

prog ::= prob { stmt1; ... stmtn; return e }
```

The block is processed right-to-left: each statement wraps the compiled continuation in the appropriate core term. The desugaring is done by using the derived sequencing bind. As an example, the medical-test program from Section 2 is written in prob-block style as in Section 2.2

6 Denotational Semantics

To define our denotational semantics, we can start with type interpretation. Each value type α is interpreted as a measurable space $(\llbracket \alpha \rrbracket, \Sigma_\alpha)$, where Σ_α is the discrete σ -algebra making every subset measurable.

$$\begin{aligned} \llbracket \text{Bool} \rrbracket &= \{\text{true}, \text{false}\} \\ \llbracket \text{Fin } n \rrbracket &= \{0, \dots, n-1\} \\ \llbracket \alpha \times \beta \rrbracket &= \llbracket \alpha \rrbracket \times \llbracket \beta \rrbracket \end{aligned}$$

The weight type ENNReal serves as the coefficient semiring throughout. All arithmetic on coefficients—addition, scalar multiplication, subtraction (saturating), and inversion—is interpreted in $[0, \infty]$.

For distribution interpretation, the function $\llbracket \cdot \rrbracket_D : \text{Dist ENNReal } \alpha \rightarrow \text{Measure}(\llbracket \alpha \rrbracket)$ interprets each primitive distribution as a measure, defined by the following equations:

$$\begin{aligned} \llbracket \text{bernoulli } p \rrbracket_D &= p \cdot \delta_{\text{true}} + (1-p) \cdot \delta_{\text{false}} \\ \llbracket \text{uniform_fin } n \rrbracket_D &= \sum_{k=0}^{n-1} n^{-1} \cdot \delta_k \\ \llbracket \text{categorical } f \rrbracket_D &= \sum_{k=0}^{n-1} f(k) \cdot \delta_k \end{aligned}$$

where δ_a denotes the Dirac measure at a , and $\cdot$ denotes scalar multiplication of a measure by an element of $[0, \infty]$.

For the interpretation of the whole program, we define structural recursion on programs with the function $\llbracket \cdot \rrbracket : \text{Prog ENNReal } \alpha \rightarrow \text{Measure}(\llbracket \alpha \rrbracket)$:

$$\begin{aligned} \llbracket \text{ret } v \rrbracket &= \delta_v \\ \llbracket \text{sample } d \ k \rrbracket &= \llbracket d \rrbracket_D \gg (\lambda a. \llbracket k \ a \rrbracket) \\ \llbracket \text{observe } b \ e \rrbracket &= \begin{cases} \llbracket e \rrbracket & \text{if } b = \text{true} \\ 0 & \text{if } b = \text{false} \end{cases} \\ \llbracket \text{score } r \ e \rrbracket &= r \cdot \llbracket e \rrbracket \end{aligned}$$

where $\gg$ denotes the measure-theoretic bind of Section 3. The bind derived operation satisfies:

$$\llbracket e \gg= k \rrbracket = \llbracket e \rrbracket \gg (\lambda a. \llbracket k \ a \rrbracket)$$

which follows directly from the structural recursion equations of bind (Section 5) and the cases above.

For use in compositional reasoning, every closed program is also interpreted as a constant kernel. Given a context type Γ , the kernel interpretation is:

$$\llbracket e \rrbracket_K : \text{Kernel}(\llbracket \Gamma \rrbracket, \llbracket \alpha \rrbracket), \quad \llbracket e \rrbracket_K(\gamma) = \llbracket e \rrbracket \quad \text{for all } \gamma : \Gamma.$$

Since the program e contains no free variables from Γ , the kernel is constant: it returns the same measure $\llbracket e \rrbracket$ regardless of the context element γ . This lifting to kernels is the starting point for extending the semantics to open programs in future work.

When a program has finite and nonzero total mass—the typical case after conditioning via observe—its denotation can be normalized to a probability measure. The *evidence* of a program e is the total mass of its denotation:

$$\text{evidence}(e) = \llbracket e \rrbracket(\Omega) \in [0, \infty].$$

The *normalized posterior* is:

$$\text{normalize}(e) = \text{evidence}(e)^{-1} \cdot \llbracket e \rrbracket.$$

Theorem (Normalization). If $0 < \text{evidence}(e) < \infty$, then $\text{normalize}(e)$ is a probability measure.

The proof reduces to the scalar identity $c^{-1} \cdot c = 1$ in $[0, \infty]$, which holds whenever $c \in (0, \infty)$.

7 Verified Properties

This section presents the machine-checked theorems established in our project.

7.1 Monad Laws

The interpretation $\llbracket \cdot \rrbracket$ is a monad morphism: it commutes with the monadic structure of $\text{Prog } P$ and the measure monad. Three laws are verified.

1. Left unit: $\llbracket (\text{ret } a) \gg= f \rrbracket = \llbracket f \ a \rrbracket$.
2. Right unit: $\llbracket e \gg= \text{ret} \rrbracket = \llbracket e \rrbracket$.
3. Associativity: $\llbracket (e \gg= f) \gg= g \rrbracket = \llbracket e \gg= (\lambda a. f \ a \gg= g) \rrbracket$.

The proof for the first properties is quite straightforward, while the other two require structural induction.

7.2 Bind Commutativity

The central result of ProbaLean, following [4], is that two independent sampling steps commute. The theorem is as follows: Let γ be a measurable space, $t : \text{Prog ENNReal } \alpha$, $u : \text{Prog ENNReal } \beta$, and $v : \alpha \rightarrow \beta \rightarrow \text{Prog ENNReal } \gamma$. Then:

$$\llbracket t >>= (\lambda a. u >>= (\lambda b. v a b)) \rrbracket = \llbracket u >>= (\lambda b. t >>= (\lambda a. v a b)) \rrbracket.$$

This is also done via structural induction. Since we can only express discrete probabilistic distributions in our language, the lemma that we could swap two sums is very easy to derive: Let ι_1 and ι_2 be finite types, γ a measurable space, $w_1 : \iota_1 \rightarrow [0, \infty]$, $w_2 : \iota_2 \rightarrow [0, \infty]$, and $f : \iota_1 \rightarrow \iota_2 \rightarrow \text{Measure}(\gamma)$. Then:

$$\sum_i w_1(i) \cdot \sum_j w_2(j) \cdot f(i, j) = \sum_j w_2(j) \cdot \sum_i w_1(i) \cdot f(i, j).$$

7.3 Soundness Theorem

The soundness theorem establishes that our denotational semantics agrees with our operational semantics; that is, running a program and converting the result into a measure yields the same measure as interpreting the program denotationally.

The main difficulty of this proof is a type mismatch: the operational semantics compute with types that support evaluation, in our case Float and $\mathbb{Q}_{\geq 0}$. The denotational semantics, however, is built purely on Mathlib’s ENNReal , which supports all the measure-theoretic machinery we need but is inherently noncomputable. This necessitates the *coerce* operation from Section 5, which lets us lift a program from one weight type to another by applying the conversion function $\text{conv} : P \rightarrow [0, \infty]$. To ensure that this conversion is sound, we require that conv is a semiring homomorphism, in particular that it preserves the unit, multiplication, complement, and uniform weights (e.g. $\text{conv}(1/n) = n^{-1}$). We also use a helper function $\text{listToMeasure conv}$ to convert the output of the evaluation with our measure-based representations. We now provide a proof sketch of the theorem.

Theorem 7.1. *Let α be a measurable space, ops an arithmetic over type P , and $\text{conv} : P \rightarrow [0, \infty]$ a semiring homomorphism. Then for any program $e : \text{Prog } P \ \alpha$:*

$$\llbracket \text{coerce conv } e \rrbracket = \text{listToMeasure conv } (e.\text{evalWith ops})$$

Proof Sketch. Proceed by structural induction on e . The *ret* case is immediate, as both sides reduce to a single Dirac measure. Similarly, the *observe* and *score* cases follow directly from the inductive hypothesis, since *observe* maps to an if-then-else on both sides, and *score* maps to scalar multiplication on both sides.

The more involved case is *sample d k*. On the denotational side, this becomes a measure bind $\llbracket d \rrbracket_D \gg (\lambda a. \llbracket k a \rrbracket)$;

on the operational side, it becomes a *flatMap* over the distributions with weights multiplied in. Because all our distribution types are finite, the measure bind reduces to a finite weighted sum. Each term pairs a sampling weight with the measure from the corresponding continuation. Our semiring homomorphism conditions then guarantee that converting the weights commutes with the arithmetic. In particular, products of weights on the operational side correspond to scalar multiplication on the measure side, and the Bernoulli complement $1 - p$ is preserved. The inductive hypothesis then closes the proof for each distribution type.

We obtain concrete instantiations of the soundness proof at three weight types. For ENNReal , the conversion is the identity and the homomorphism conditions hold trivially. For Float , we are forced to axiomatize the conversion and the homomorphism properties since Lean’s Float has no formal algebraic semantics. The most satisfying instantiation is $\mathbb{Q}_{\geq 0}$. The non-negative rationals are fully computable, so programs can be executed; however, unlike Float , we can directly construct a conversion to ENNReal that satisfies the homomorphism properties. This gives us a weight type where the entire pipeline, from execution to denotational correctness, is machine-checked with no axioms beyond Lean’s kernel.

8 Conclusion

This paper has presented ProbaLean, a machine-checked formalization of a discrete probabilistic programming language in Lean 4. Starting from a small free-monad syntax with four constructors—*ret*, *sample*, *observe*, and *score*—ProbaLean develops formal denotational semantics that assigns s -finite measures to programs using Mathlib’s measure-theory library. All the central theoretical contributions are all machine-checked, except in only one case about Float depending on postulated axioms.

Two limitations bound the scope of the current work. First, every distribution has finite discrete support; extending to continuous distributions would require replacing finite-sum expansions with Lebesgue integration, a substantial change to the semantic architecture. Second, the Float soundness result relies on postulated axioms. These limitations aside, the formalization demonstrates that Lean 4 and Mathlib together provide a viable foundation for machine-checked probabilistic programming semantics.

References

- [1] 2020. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs* (New Orleans, LA, USA) (*CPP 2020*). Association for Computing Machinery, New York, NY, USA, 367–381. <https://doi.org/10.1145/3372885.3373824>
- [2] Reynald Affeldt, Cyril Cohen, and Ayumu Saito. [n.d.]. Semantics of Probabilistic Programs Using s -Finite Kernels in Dependent Type Theory. 1, 3 ([n.d.]), 16:1–16:34. <https://doi.org/10.1145/3732291>

- [3] Michèle Giry. 1982. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, B. Banaschewski (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 68–85.
- [4] Sam Staton. [n.d.]. Commutative Semantics for Probabilistic Programming. In *Programming Languages and Systems* (Berlin, Heidelberg, 2017), Hongseok Yang (Ed.). Springer, 855–879. https://doi.org/10.1007/978-3-662-54434-1_32